

91. Une struct : ce qui va ensemble

La position, la direction et les vies d'un héros ne sont pas trois choses : c'est UN héros.

1. Ce qu'il faut comprendre

`struct Heros { uint8_t x; uint8_t y; uint8_t vies; };` **invente un type**. Il ne réserve rien : c'est un **plan**.

`Heros joueur;` construit une variable de ce type — trois octets qui se suivent.

On atteint chaque **champ** avec un point : `joueur.x`, `joueur.vies`. Ce sont des variables ordinaires :

`joueur.x++`, `if (joueur.y > 2)`, tout marche.

Pourquoi pas trois variables `herosX`, `herosY`, `herosVies` ? Pour un héros, ce serait pareil. Mais dès qu'il y en a **deux** — un second joueur, un ennemi — les noms se multiplient, et rien ne dit plus que `herosX` et `herosVies` parlent de la même chose. La `struct` le **dit** : c'est le code qui porte le sens.

Ici, le héros se déplace dans les quatre directions, en gardant sa position d'avant pour **effacer** l'ancienne case — et seulement s'il a bougé (la règle du VBlank). Un déplacement ne se fait qu'un tour sur quatre, grâce au compteur `attente` — sinon il traverserait l'écran en un tiers de seconde.

À toi : ajoute un champ `pas` (le nombre de pas faits), augmente-le à chaque déplacement, et affiche-le.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **814 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	257 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← texte() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
CopierTuiles	76 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	256 o	← CopierTuiles — 16 lettres sur 43
"VIES:"	5 o	← texte() — un texte gravé
"#"	1 o	← texte() — un texte gravé
" "	1 o	← texte() — un texte gravé
"#"	1 o	← texte() — un texte gravé

Lettres gravées : EISV0123456789:#

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 4    77 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
bouton() × 4    20 o sur place, dans main() + LireManette (37 o)
nombre() × 1    11 o sur place, dans main() + EcrireNombre (33 o)
image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : un héros # qui se déplace avec les 4 flèches.
// Sa position et ses vies sont rangées ENSEMBLE, dans une struct.
//
// Ce qui est nouveau : la STRUCT.
// struct Heros ... ; = on invente un nouveau type, « Heros »,
//                     qui contient plusieurs cases (x, y, vies).
// Heros joueur;      = on crée une variable de ce type.
// joueur.x            = la case x DE joueur (le point veut dire « de »).

#include <texte>        // écrit un texte à l'écran
#include <nombre>       // écrit un nombre en chiffres
#include <bouton>       // lit un bouton de la manette
```

2. UN ENREGISTREMENT — HEROS

```
struct Heros {          // Le modèle d'un héros :
    uint8_t x;           // sa colonne,
    uint8_t y;           // sa ligne,
    uint8_t vies;        // ses vies.
};                       // (Le point-virgule après l'accolade est obligatoire.)
```

3. UNE VARIABLE — JOUEUR

```
Heros joueur;           // UN héros, nommé joueur.
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    joueur.x = 9;                            // On remplit ses cases.
    joueur.y = 9;
    joueur.vies = 3;

    texte(1, 1, "VIES:");
    nombre(7, 1, joueur.vies, 1); // Affiche 3.
    texte(joueur.x, joueur.y, "#"); // Dessine le héros en (9, 9).

    uint8_t attente = 0;                    // Pour ralentir : on ne bouge qu'une image sur 4.

    while (true) {                          // La boucle du jeu :
        image();

        attente++;
        if (attente < 4) continue;          // un tour sur quatre : sinon on saute la suite du tour
        attente = 0;

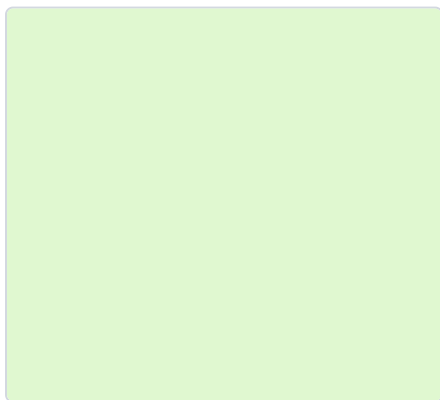
        uint8_t ancienX = joueur.x; // On retient la place d'avant...
        uint8_t ancienY = joueur.y;

        // ... puis on bouge, sans sortir de l'écran :
        if (bouton(DROITE) && joueur.x < 19) joueur.x++; // droite, pas plus loin que 19
        if (bouton(GAUCHE) && joueur.x > 0)  joueur.x--; // gauche, pas moins que 0
        if (bouton(BAS) && joueur.y < 17)   joueur.y++; // bas : y grandit vers le bas
        if (bouton(HAUT) && joueur.y > 3)   joueur.y--; // haut, pas au-dessus de la ligne 3
                                                // (les lignes 0 à 2 sont pour VIES)

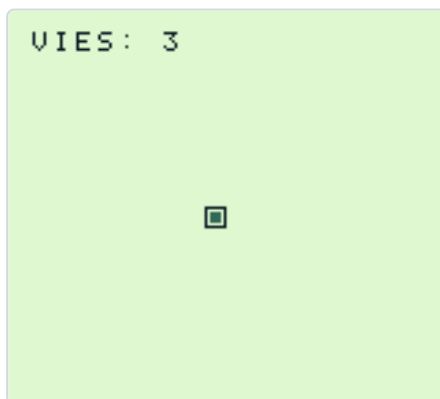
        if (joueur.x != ancienX || joueur.y != ancienY) { // A-t-il bougé ?
            texte(ancienX, ancienY, " ");                // efface l'ancienne place,
            texte(joueur.x, joueur.y, "#");              // dessine la nouvelle.
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



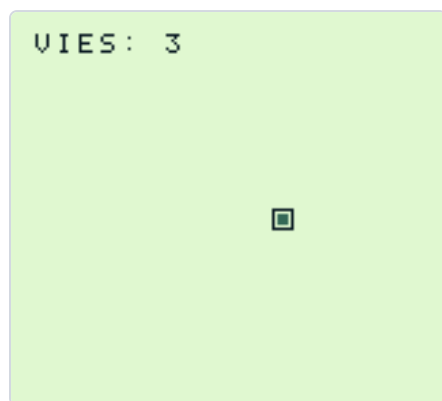
la toute première image — la console vient de démarrer — image 1



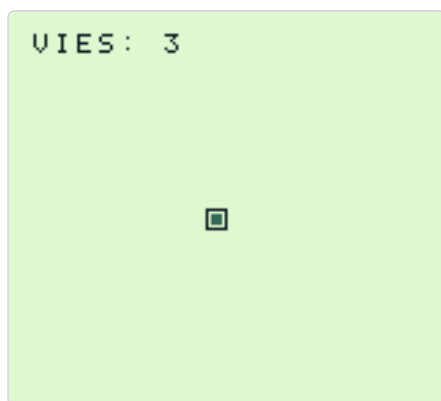
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

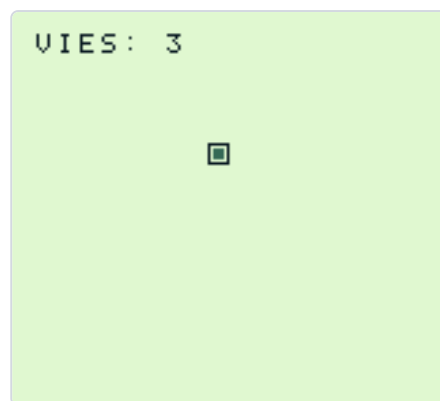
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



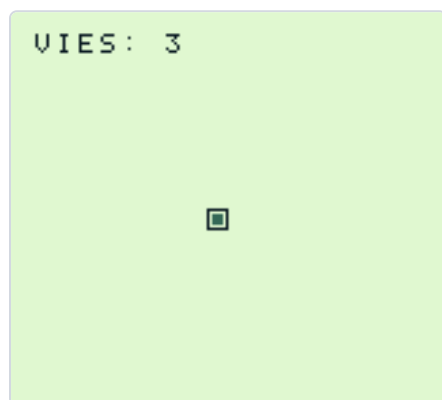
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Un # au milieu de l'écran, que les quatre flèches promènent, et « VIES: 3 » en haut.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les vies sont affichées
- ✓ le héros part de $x = 9$
- ✓ un seul # à l'écran : l'ancienne case est effacée (1)
- ✓ il s'est déplacé vers la droite et vers le bas (13, 16)

Ensuite : **92. Un tableau de struct, parcouru par une boucle** — chapitre 14 : Les struct : ce qui va ensemble. · gameboy3 — écrire une cartouche Game Boy en C++